Claude Code Deep Dive Whitepaper

From Beginner to Expert: Architecture Analysis, Internals & Power User Guide

Version: Based on source code analysis of Claude Code v2.1.88
Date: April 2026
Source: In-depth reverse engineering analysis based on the public repositories `sanbuphy/claude-code-source-code` and `ChinaSiro/claude-code-sourcemap`

Part 1: Overview & Background

1.1 What Is Claude Code

Claude Code is Anthropic's official AI-powered CLI (Command Line Interface) tool for programming. It is far more than a simple chat interface — it is a **full-fledged agent framework**, featuring:

- **Full file system access** — read, edit, and create files
- **Shell command execution** — run arbitrary Bash/PowerShell commands
- **Multi-agent coordination** — the main agent can spawn sub-agents to work in parallel
- **MCP protocol integration** — extend tool capabilities through a standard protocol
- **Intelligent context management** — automatic compaction and token budget tracking
- **Permission security system** — multi-layered permission model to prevent accidental operations
- **Hook event system** — users can inject custom logic at key execution points
- **Session persistence** — resumable, interruptible long-running sessions

1.2 Technical Scale

Metric	Value
TypeScript source files	**1,884**
Total lines of code	**~513,000 LOC**
Tool implementations	**45+**
Slash commands	**80+**
Service modules	**38**
MCP transport protocols supported	**6**
Hook event types	**15+**
Feature gate modules	**108**

1.3 Technology Stack

```
Language:       TypeScript (ES2022 target)
Runtime:        Node.js >= 18 / Bun (internal)
UI Framework:   React + Ink (terminal UI rendering)
```

```
Build Tool:     esbuild + custom transform pipeline
Package Mgmt:   npm (ESM modules)
API:            Anthropic Messages API
Protocol:       MCP (Model Context Protocol)
```

Part 2: Project Structure Overview

2.1 Top-Level Directory Structure

```
claude-code/
├── src/                          # Main application source (1,884 file
│   ├── entrypoints/              # Entry points
│   │   └── cli.tsx               # CLI main entry (~300 lines)
│   ├── main.tsx                  # Main orchestrator (~4,683 lines)
│   ├── query.ts                  # Core query loop (~1,729 lines) ★ Mo
│   ├── QueryEngine.ts            # Query engine wrapper
│   ├── Tool.ts                   # Tool base interface
│   ├── tools.ts                  # Tool registry
│   ├── commands.ts               # Command registry
│   ├── context.ts                # Context assembly
│   │
│   ├── tools/                    # Tool implementations (~45)
│   │   ├── AgentTool/            # Agent tool (17 files, 400KB)
│   │   ├── BashTool/             # Bash execution
│   │   ├── FileEditTool/         # File editing (8 files)
│   │   ├── FileReadTool/         # File reading
│   │   ├── FileWriteTool/        # File writing
│   │   ├── GlobTool/             # File search
│   │   ├── GrepTool/             # Content search
│   │   ├── WebFetchTool/         # Web fetching
│   │   ├── WebSearchTool/        # Web search
│   │   ├── MCPTool/              # MCP tool bridge
│   │   ├── NotebookEditTool/     # Jupyter editing
│   │   ├── SkillTool/            # Skill invocation
│   │   └── ...                   # More tools
│   │
│   ├── commands/                 # Slash commands (~80)
│   │   ├── commit.ts             # /commit
│   │   ├── review.ts             # /review
│   │   ├── mcp/                  # /mcp management
│   │   ├── config/               # /config configuration
│   │   ├── tasks/                # /tasks management
│   │   └── ...
│   │
│   ├── services/                 # Business services (38 directories)
│   │   ├── api/                  # API communication layer
│   │   │   ├── claude.ts         # Anthropic API client (125KB)
│   │   │   ├── withRetry.ts      # Retry logic (28KB)
│   │   │   └── errors.ts         # Error handling
│   │   ├── mcp/                  # MCP service (25 files, 900KB)
│   │   │   ├── client.ts         # MCP client (119KB)
```

```
│   │   │   ├── config.ts          # MCP configuration
│   │   │   ├── auth.ts            # OAuth authentication (88KB)
│   │   │   └── types.ts           # Type definitions
│   │   ├── analytics/             # Telemetry & analytics
│   │   ├── compact/               # Context compaction
│   │   ├── tools/                 # Tool execution service
│   │   │   └── StreamingToolExecutor.ts  # Streaming tool executor
│   │   └── ...
│   │
│   ├── utils/                     # Utility functions (30+ groups)
│   │   ├── permissions/           # Permission system (26 files, 720KB)
│   │   ├── hooks/                 # Hook execution
│   │   ├── settings/              # Configuration management (50+ files
│   │   ├── config.ts              # Configuration hierarchy
│   │   ├── claudemd.ts            # CLAUDE.md parsing
│   │   ├── tokens.ts              # Token counting
│   │   └── ...
│   │
│   ├── components/                # React/Ink terminal UI components (4
│   ├── state/                     # Application state management
│   ├── types/                     # Type definitions
│   ├── hooks/                     # React Hooks
│   ├── constants/                 # Constants & prompt templates
│   │   ├── prompts.ts             # System prompt construction
│   │   └── systemPromptSections.ts  # Prompt caching
│   ├── skills/                    # Skill system
│   ├── tasks/                     # Task management
│   │
│   ├── coordinator/               # Multi-agent coordination (Feature-g
│   ├── assistant/                 # KAIROS autonomous mode
│   ├── bridge/                    # Remote control
│   ├── daemon/                    # Background daemon
│   ├── remote/                    # Remote triggers
│   ├── voice/                     # Voice features
│   └── vim/                       # Vim integration
│
├── scripts/                       # Build scripts
│   ├── build.mjs                  # Main build pipeline
│   ├── prepare-src.mjs            # Source preprocessing
│   └── transform.mjs              # Code transformations
├── vendor/                        # Native module stubs
├── stubs/                         # Bun compile-time internal feature s
├── tools/                         # External tools (ripgrep, etc.)
├── package.json                   # v2.1.88
└── tsconfig.json                  # ES2022 configuration
```

2.2 Key Files Ranked by Size

Rank	File	Size	Purpose
1	`main.tsx`	803 KB	Main orchestrator, REPL startup

2	`tools/AgentTool/AgentTool.tsx`	233 KB	Agent tool core
3	`services/api/claude.ts`	125 KB	API communication client
4	`services/mcp/client.ts`	119 KB	MCP client
5	`services/mcp/auth.ts`	88 KB	MCP OAuth authentication
6	`query.ts`	68 KB	Core query loop
7	`utils/permissions/permissions.ts`	52 KB	Permission engine
8	`utils/permissions/yoloClassifier.ts`	52 KB	Auto mode classifier
9	`utils/analyzeContext.ts`	42 KB	Context analysis
10	`services/mcp/config.ts`	51 KB	MCP configuration management

Part 3: Core Architecture Deep Dive

3.1 Layered System Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                          CLI Entry Layer                             │
│                  (cli.tsx → main.tsx → REPL)                         │
├──────────────────────────────────────────────────────────────────────┤
│                         Command Dispatch                             │
│              (/commands → processUserInput)                          │
├──────────────────────────────────────────────────────────────────────┤
│                          Query Engine                                │
│                (QueryEngine.ts → query.ts)                           │
│      ┌──────────────────────────────────────────────┐                │
│      │      Core Agent Loop (async generator)       │                │
│      │   ┌──────────┐   ┌──────────┐   ┌──────────┐ │               │
│      │   │API Call  │→ │Tool Exec │→ │Context   │ │               │
│      │   │claude.ts │   │Streaming │   │Mgmt      │ │               │
│      │   │          │   │ToolExec  │   │compact   │ │               │
│      │   └──────────┘   └──────────┘   └──────────┘ │               │
│      └──────────────────────────────────────────────┘                │
├──────────────────────────────────────────────────────────────────────┤
│                           Tool Layer                                 │
│    ┌────────┐  ┌────────┐  ┌────────┐  ┌──────────┐                │
│    │  Bash  │  │  File  │  │  Web   │  │   MCP    │                │
│    │  Tool  │  │  Tools │  │  Tools │  │  Bridge  │                │
│    └────────┘  └────────┘  └────────┘  └──────────┘                │
├──────────────────────────────────────────────────────────────────────┤
│                        Permission Layer                              │
│      ┌──────────────┐  ┌──────────┐  ┌──────────────┐               │
│      │   Rules      │  │Classifier│  │   Hook       │               │
│      │   Engine     │  │          │  │  Decision    │               │
│      └──────────────┘  └──────────┘  └──────────────┘               │
├──────────────────────────────────────────────────────────────────────┤
```

```
│                    State & Config                          │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐   │
│   │ AppState │  │ Settings │  │ CLAUDE.md│  │ Session  │   │
│   │  Store   │  │ Hierarchy│  │  Memory  │  │ Storage  │   │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘   │
├──────────────────────────────────────────────────────────────┤
│                   External Services                        │
│    ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐  │
│    │ Anthropic API│  │ MCP Server   │  │ GrowthBook/      │  │
│    │ (Messages)   │  │ (Stdio/SSE)  │  │ Feature Flags    │  │
│    └──────────────┘  └──────────────┘  └──────────────────┘  │
└──────────────────────────────────────────────────────────────┘
```

3.2 Startup Flow in Detail

Phase 1: Fast-Path Detection (cli.tsx)

```
┌─ CLI Entry ──────────────────────────────────────────────────┐
│                                                              │
│  1. profileCheckpoint('main_tsx_entry')                      │
│  2. startMdmRawRead()        // Start MDM read in parallel  │
│  3. startKeychainPrefetch()  // Start keychain read in      │
│                              // parallel                     │
│                                                              │
│  Fast paths (zero module loading):                           │
│  ├─ --version        → Print version and exit immediately    │
│  ├─ --dump-system-prompt → Output system prompt              │
│  ├─ --chrome-native-host → Chrome extension mode             │
│  ├─ --computer-use-mcp → Computer Use MCP server             │
│  ├─ --daemon-worker  → Daemon worker thread                  │
│  ├─ remote-control   → Remote control bridge mode            │
│  ├─ daemon           → Daemon mode                           │
│  └─ ps / logs        → Background session management         │
│                                                              │
│  ↓ No match → Load full initialization                       │
└──────────────────────────────────────────────────────────────┘
```

Phase 2: Full Initialization (main.tsx)

```
┌─ Main Initialization ──────────────────────────────────────┐
│                                                              │
│  Lines 1-200:    Imports & performance checkpoints           │
│  Lines 200-500:  OAuth config & security settings loading    │
│  Lines 500-1000: GrowthBook feature flag initialization      │
│  Lines 1000-2000: Commander.js CLI argument parsing          │
│  Lines 2000-3700: Mode selection (interactive/headless/      │
│                   Print)                                     │
│  Lines 3700+:    REPL startup & interactive loop             │
│                                                              │
│  Parallel prefetching (after first render):                  │
│  ├─ initUser()                  // User profile              │
│  ├─ getUserContext()            // CLAUDE.md files           │
```

```
│  ├─ getRelevantTips()           // Feature tips            │
│  ├─ countFilesRoundedRg()       // Project file count      │
│  ├─ refreshModelCapabilities()  // Model capability matrix │
│  ├─ settingsChangeDetector.initialize() // Config watcher  │
│  └─ skillChangeDetector.initialize()    // Skill hot-      │
│                                         // reload          │
│                                                            │
└────────────────────────────────────────────────────────────┘
```

3.3 Core Query Loop (query.ts — The Heart of the System)

This is the single most critical file in all of Claude Code. The 1,729-line `query.ts` implements the **complete agent loop**:

```
// Core state definition
type State = {
  messages: Message[]                      // Conversation history
  toolUseContext: ToolUseContext           // Tool execution context
  autoCompactTracking?: AutoCompactTrackingState
  maxOutputTokensRecoveryCount: number     // API recovery loop counter
  hasAttemptedReactiveCompact: boolean     // Whether reactive compactio
  pendingToolUseSummary?: Promise<...>     // Pending tool use summary
  stopHookActive?: boolean                // Whether the stop hook is ac
  turnCount: number                       // Current turn number
  transition?: Continue                    // Recovery path
}

// Main generator function
async function* query(params: QueryParams)
  // yields: StreamEvent | RequestStartEvent | Message | TombstoneMessa
  // returns: Terminal { ... }

async function* queryLoop(params, consumedCommandUuids)
  // Implements: Core agent loop
  // Handles: Tool execution, compaction, recovery
```

Single-Turn Execution Flow

```
User Input
    │
    ▼
┌──────────────────────────────────────────┐
│ 1. Assemble system prompt + context      │
│    ├─ Base system instructions (role,    │
│    │   capabilities)                     │
│    ├─ Tool definitions + MCP tool hints  │
│    ├─ Permission rules (auto mode)       │
│    ├─ Memory files (CLAUDE.md)           │
│    └─ User/system context                │
├──────────────────────────────────────────┤
│ 2. Normalize messages → API format       │
├──────────────────────────────────────────┤
│ 3. Call claude.ts streaming API          │
```

```
│     ├─ text_delta events                    │
│     ├─ content_block_start/delta/stop       │
│     └─ Collect into complete message        │
├──────────────────────────────────────────────┤
│ 4. Check stop_reason                         │
│     ├─ "end_turn" → Exit loop               │
│     ├─ "tool_use" → Enter tool execution    │
│     └─ "max_tokens" → Recovery mechanism    │
├──────────────────────────────────────────────┤
│ 5. Tool Execution (StreamingToolExecutor)   │
│     ├─ Extract tool_use blocks               │
│     ├─ Permission check (canUseTool)         │
│     ├─ Execute tools in parallel             │
│     ├─ Stream progress updates               │
│     └─ Collect tool_result blocks            │
├──────────────────────────────────────────────┤
│ 6. Context Management                        │
│     ├─ Check token budget                    │
│     ├─ Trigger auto-compaction?              │
│     └─ Update message array                  │
├──────────────────────────────────────────────┤
│ 7. Continuation decision (7 branches)        │
│     ├─ Normal tool loop continues            │
│     ├─ Token budget exceeded → auto-         │
│     │   continue                             │
│     ├─ max_output_tokens → upgrade retry     │
│     ├─ Reactive compaction triggered          │
│     ├─ Stop hook needs execution             │
│     ├─ Context collapse triggered             │
│     └─ Task budget exhausted                 │
└──────────────────────────────────────────────┘
    │
    ▼
  Back to step 1 (or end)
```

3.4 Three Execution Modes

Mode	Trigger	Characteristics
Interactive REPL	Default startup	Full TUI, persistent session, interruptible
Headless/Print mode	`-p` / `--print` flag	Single query, output to stdout
SDK/Non-interactive mode	`--init-only`	Minimal initialization, no user prompts

---

Part 4: Tool System

4.1 Tool Interface Definition (Tool.ts)

```
// Tool definition type
type Tool = {
  name: string                                  // Tool name
```

```
  description: string                        // Description for the mode
  input_schema: ToolInputJSONSchema          // JSON Schema input spec
  isEnabled?: () => boolean                  // Whether enabled
  isIncludedInSystemPrompt?: () => boolean   // Whether in system prompt
  validateInput?: (input) => ValidationResult  // Input validation
  expandedDescription?: string               // Extended description
  visibility?: 'public' | 'internal'         // Visibility
  mcpInfo?: { serverName; toolName }         // MCP information
}

// Tool execution context
type ToolUseContext = {
  options: {
    commands: Command[]
    tools: Tools
    mcpClients: MCPServerConnection[]
    agentDefinitions: AgentDefinitionsResult
    thinkingConfig: ThinkingConfig
    maxBudgetUsd?: number
    // ... 20+ fields
  }
  abortController: AbortController
  readFileState: FileStateCache
  getAppState(): AppState
  setAppState(f: (prev) => AppState): void
}

// Tool result type
type ToolResult<Output> = {
  content: ContentBlockParam[]    // API-format content
  successMessage?: string         // Success message
  toolUseID?: string             // Tool use ID
  contentReplaced?: boolean       // Whether content was replaced
  suppressOutput?: boolean        // Whether to hide output
  // ... 15+ more fields
}
```

4.2 Complete Tool Inventory

File Operations (6)

Tool	Function	Key Features
FileRead	Read files	Supports images/PDF/Jupyter, line number ranges
FileWrite	Create/overwrite files	Must read before overwriting
FileEdit	Precise string replacement	`old_string` → `new_string`, supports `replace_all`
Glob	File pattern matching	Supports glob patterns like `**/*.ts`
Grep	Content search	Based on ripgrep, supports regex, context lines
NotebookEdit	Jupyter editing	Edit `.ipynb` cells

Execution (5)

Tool	Function	Key Features
Bash	Shell command execution	Sandbox support, timeout control, background execution
PowerShell	Windows commands	Windows platform only
REPL	Interactive REPL	Internal only (ANT-only)
Skill	Skill invocation	Execute predefined/user-defined skills
Agent	Agent spawning	Create sub-agents for parallel work

Networking (3)

Tool	Function	Key Features
WebFetch	Fetch web content	URL access, content extraction
WebSearch	Web search	Search engine integration
WebBrowser	Browser automation	Feature-gated, automated operations

MCP Bridge (4)

Tool	Function	Key Features
MCPTool	MCP tool proxy	Bridges MCP server tools
ListMcpResources	List MCP resources	Discover available resources
ReadMcpResource	Read MCP resources	Fetch resource contents
ListPeers	List UDS peers	Internal communication

Task Management (6)

Tool	Function
TaskCreate	Create a task
TaskGet	Get task details
TaskUpdate	Update task status
TaskList	List all tasks
TaskStop	Stop a task
TaskOutput	Get task output

System (5)

Tool	Function
AskUserQuestion	Ask the user a question
ToolSearch	Search for lazily loaded tools
EnterPlanMode	Enter planning mode
ExitPlanMode	Exit planning mode
ConfigTool	Configuration management

4.3 Tool Registration Mechanism (tools.ts)

```
// Dynamically build the available tool list
export function getAllBaseTools(): Tools {
  return [
    AgentTool,
    TaskOutputTool,
    BashTool,
    ...(hasEmbeddedSearchTools() ? [] : [GlobTool, GrepTool]),
    ExitPlanModeV2Tool,
    FileReadTool,
    FileEditTool,
    FileWriteTool,
    NotebookEditTool,
    WebFetchTool,
    TodoWriteTool,
    WebSearchTool,
    TaskStopTool,
    AskUserQuestionTool,
    SkillTool,
    // Feature-gated conditional tools
    ...(SleepTool ? [SleepTool] : []),
    ...(cronTools ? cronTools : []),
    ...(RemoteTriggerTool ? [RemoteTriggerTool] : []),
    // ...
  ]
}

// Filter tools by deny rules
export function filterToolsByDenyRules<T extends { name: string }>(
  tools: readonly T[],
  permissionContext: ToolPermissionContext
): T[] {
  return tools.filter(tool => !getDenyRuleForTool(permissionContext, to
}
```

4.4 Tool Execution Pipeline (StreamingToolExecutor)

```
Claude API returns tool_use blocks
         |
         ▼
┌──────────────────────────┐
| Extract all tool_use     |
| blocks (may be multiple  |
| in parallel)             |
└──────────┬───────────────┘
           |
    ┌──────┴──────┐
    ▼             ▼
┌──────────┐  ┌──────────┐
| Perm.    |  | Perm.    |   (parallel)
```

```
│ Check    │   │ Check    │
│ Tool A   │   │ Tool B   │
└──────────┘   └──────────┘
     │              │
     ▼              ▼

┌──────────┐   ┌──────────┐
│ Execute  │   │ Execute  │    (parallel, with
│ Tool.A   │   │ Tool.B   │     concurrency control)
│ .call()  │   │ .call()  │
└──────────┘   └──────────┘
     │              │
     ▼              ▼

┌──────────────────────────┐
│ Collect tool_result      │
│ blocks, append to        │
│ message array            │
└──────────────────────────┘
```

Part 5: Permissions & Security Model

5.1 Permission Modes

Claude Code provides **5 external permission modes** and **2 internal permission modes**:

```
External modes (user-selectable):
├── default          → Interactive prompt (default)
├── acceptEdits      → Auto-approve file edits
├── bypassPermissions → Dangerous: approve all without prompting
├── dontAsk          → Silently deny
└── plan             → Planning mode (/plan command only)

Internal modes (system use):
├── auto             → Classifier-driven decisions (Feature-gated)
└── bubble           → Internal state
```

5.2 Permission Decision Flow

```
Tool call request
     │
     ▼

┌──────────────────────┐
│ 1. Check Allow rules│ ← alwaysAllowRules in settings.json
│    Match? → Allow    │
└──────────────────────┘
           │ No match
           ▼

┌──────────────────────┐
│ 2. Check Deny rules  │ ← alwaysDenyRules in settings.json
│    Match? → Deny     │
└──────────────────────┘
           │
```

```
          | No match
          ▼
┌──────────────────────┐
| 3. Check Ask rules   | ← alwaysAskRules in settings.json
|    Match? → Prompt   |
└──────────────────────┘
          | No match
          ▼
┌──────────────────────┐
| 4. Check perm. mode |
|    bypass → Allow    |
|    dontAsk → Deny    |
|    auto → Classifier|
|    default → Prompt  |
└──────────────────────┘
          | auto mode
          ▼
┌──────────────────────┐
| 5. YOLO Classifier   |
|    safe → Allow      |
|    unsafe → Prompt   |
|    unknown → Prompt  |
└──────────────────────┘
```

5.3 Permission Rule Format

```
{
  "permissions": {
    "rules": {
      "allow": [
        "Bash(prefix:npm test)",
        "Bash(prefix:git status)",
        "FileEdit",
        "FileRead",
        "Glob",
        "Grep"
      ],
      "deny": [
        "Bash(prefix:rm -rf /)",
        "Bash(prefix:sudo)"
      ]
    }
  }
}
```

Rule matching syntax:

- `ToolName` — matches the entire tool
- `ToolName(prefix:xxx)` — matches tool name + argument prefix
- `MCP:ServerName` — matches an MCP server

5.4 YOLO Classifier (Core of Auto Mode)

```
// yoloClassifier.ts — analyzes command safety
classifyYoloAction(command: string, cwd: string)
  → 'safe' | 'unsafe' | 'unknown'

// Dangerous pattern detection:
// ├─ File system operations outside the working directory
// ├─ Credential/environment variable exposure
// ├─ System administration commands
// ├─ Network operations
// ├─ rm -rf, git reset --hard, git push --force
// ├─ drop, delete, truncate
// └─ sudo, --no-verify
```

5.5 Sandbox Support

```
// sandbox-adapter.ts — Bash sandboxing
SandboxManager.isSandboxingEnabled()
SandboxManager.areUnsandboxedCommandsAllowed()
SandboxManager.isAutoAllowBashIfSandboxedEnabled()

// Capability restrictions:
// ├─ File access whitelist/blacklist
// ├─ Network restrictions
// └─ Environment variable filtering
```

Part 6: Hook Event System

6.1 Hook Event Types

Claude Code provides **15+ hook event types**, allowing users to inject custom logic at key execution points:

Event	Trigger Timing	Typical Use Case
`SessionStart`	Session begins	Initialize environment, load config
`UserPromptSubmit`	Before user input is submitted	Input validation, logging
`PreToolUse`	Before tool execution	Permission control, parameter modification
`PostToolUse`	After tool execution	Result processing, logging
`PostToolUseFailure`	After tool execution failure	Error handling, alerts
`PermissionRequest`	Before permission dialog	Automatic decision-making
`PermissionDenied`	User denies permission	Record denial reason
`Notification`	Notification sent	Custom notification channels
`Setup`	First-run setup	Environment preparation
`Elicitation`	URL fetch request	MCP URL handling
`CwdChanged`	Working directory changed	Auto-reload configuration

`FileChanged`	Watched file changed	Hot reload
`SubagentStart`	Sub-agent starts	Track/limit sub-agents
`WorktreeCreate`	Worktree created	Isolated environment management

6.2 Hook Configuration

Three types of hooks can be configured in `settings.json`:

```
{
  "hooks": [
    {
      "event": "PreToolUse",
      "matcher": "Bash",

      "handler": {
        "type": "script",
        "command": "~/.claude/hooks/check-bash.sh"
      }
    },
    {
      "event": "PostToolUse",

      "handler": {
        "type": "javascript",
        "code": "module.exports = async (input) => { /* ... */ }"
      }
    },
    {
      "event": "UserPromptSubmit",

      "handler": {
        "type": "agent",
        "prompt": "Validate whether the user input is safe...",
        "timeout": 30,
        "model": "claude-sonnet-4-6"
      }
    }
  ]
}
```

6.3 Hook Response Protocol

```
// Synchronous hook response
{
  continue?: boolean,           // Whether to continue execution
  suppressOutput?: boolean,     // Hide output
  stopReason?: string,          // Stop reason
  decision?: 'approve' | 'block',  // Approval decision
  reason?: string,              // Decision reason
  systemMessage?: string,       // Warning message
```

```
  hookSpecificOutput?: {
    hookEventName: string,
    permissionDecision?: 'allow' | 'deny' | 'ask',
    updatedInput?: Record<string, unknown>,  // Modified input
    updatedMCPToolOutput?: unknown,
    watchPaths?: string[],
    additionalContext?: string
  }
}

// Asynchronous hook response
{
  async: true,
  asyncTimeout?: number    // Timeout in seconds
}
```

6.4 Hook Practical Scenarios

Scenario 1: Auto-approve safe commands

```
{
  "event": "PreToolUse",
  "matcher": "Bash",
  "handler": {
    "type": "script",
    "command": "echo '{\"hookSpecificOutput\":{\"permissionDecision\":\
  }
}
```

Scenario 2: Log all tool calls

```
{
  "event": "PostToolUse",
  "handler": {
    "type": "javascript",
    "code": "module.exports = async ({tool, input, output}) => { requir
  }
}
```

Scenario 3: AI-powered code change review

```
{
  "event": "PostToolUse",
  "matcher": "FileEdit",
  "handler": {
    "type": "agent",
    "prompt": "Review this file edit for security vulnerabilities. Chec
    "timeout": 60
  }
}
```

Part 7: MCP Protocol Integration

7.1 What Is MCP

MCP (Model Context Protocol) is an open standard protocol that enables AI applications to interact with external data sources and tools. Claude Code includes a complete built-in MCP client implementation.

7.2 Supported Transport Protocols

Protocol	Type	Use Case
Stdio	Process communication	Local tool servers
SSE	HTTP Server-Sent Events	Remote servers
HTTP	Request/Response	REST API integration
WebSocket	Bidirectional communication	Real-time interactive services
SDK	In-process	Internal integration
ClaudeAI Proxy	Proxy	Claude.ai cloud

7.3 MCP Server Configuration

```
// ~/.claude/mcp.json or mcpServers in settings.json
{
  "mcpServers": {
    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/path/
    },
    "github": {
      "type": "sse",
      "url": "https://mcp.github.com/sse",
      "headers": {
        "Authorization": "Bearer ${GITHUB_TOKEN}"
      }
    },
    "database": {
      "type": "http",
      "url": "https://db-mcp.example.com/mcp",
      "oauth": {
        "clientId": "xxx",
        "authServerMetadataUrl": "https://auth.example.com/.well-known/
      }
    }
  }
}
```

7.4 MCP Tool Integration Flow

```
At startup
  |
  ▼

┌──────────────────────────────────┐
| 1. Discovery: List all MCP tools|
|    client.tools/list             |
├──────────────────────────────────┤
| 2. Integration: Wrap as Claude   |
|    tools                         |
|    MCPTool ← { serverName,       |
|               toolName,          |
|               inputSchema }      |
├──────────────────────────────────┤
| 3. Registration: Add to main     |
|    tool registry                 |
|    getAllBaseTools() + mcpTools  |
├──────────────────────────────────┤
| 4. Execution: Route to MCP when  |
|    model invokes tool            |
|    MCPTool.call(input)           |
|    → client.tools/call           |
|    → Deserialize result          |
|    → Return ToolResult           |
├──────────────────────────────────┤
| 5. Streaming: Support server-    |
|    push responses                |
|    SSE/WebSocket continuous      |
|    receive                       |
└──────────────────────────────────┘
```

7.5 MCP Resource Prefetching

```
// Asynchronously prefetch all MCP resources
prefetchAllMcpResources(mcpClients: MCPServerConnection[]): Promise<voi
  // Parallelize resource/list calls
  // Cache results for subsequent access
  // Gracefully handle timeouts
```

Part 8: Configuration System

8.1 Four-Level Configuration Hierarchy

```
Priority from highest to lowest:

┌──────────────────────────────────────────┐
| 1. Managed Settings (highest priority)   |
|    /etc/claude-code/settings.json        |
|    macOS MDM policies                    |
|    Remote API polling control            |
├──────────────────────────────────────────┤
```

```
│ 2. User Settings                          │
│    ~/.claude/settings.json                │
│    Global defaults                        │
├──────────────────────────────────────────┤
│ 3. Project Settings                       │
│    CLAUDE.md (project root)               │
│    .claude/CLAUDE.md                      │
│    .claude/rules/*.md                     │
├──────────────────────────────────────────┤
│ 4. Runtime Settings (lowest priority)     │
│    CLI flags (--permission-mode, etc.)    │
│    Environment variables                  │
│    (CLAUDE_CODE_* prefix)                 │
└──────────────────────────────────────────┘
```

8.2 Complete settings.json Structure

```
type GlobalConfig = {
  // ── General ──
  numStartups: number
  userID?: string
  installMethod?: 'local' | 'native' | 'global' | 'unknown'
  hasCompletedOnboarding?: boolean

  // ── Permissions ──
  theme: ThemeSetting
  permissions?: {
    defaultMode: PermissionMode
    rules?: {
      allow?: string[]     // e.g. 'Bash(prefix:rm)', 'FileEdit'
      deny?: string[]
      ask?: string[]
    }
  }

  // ── API & Auth ──
  primaryApiKey?: string
  customApiKeyResponses?: {
    approved?: string[]
    rejected?: string[]
  }

  // ── Environment ──
  env: { [key: string]: string }

  // ── Feature Toggles ──
  autoCompactEnabled: boolean
  verbose: boolean

  // ── Notifications ──
  preferredNotifChannel: 'system' | 'email' | 'slack'
```

```
  // ── OAuth ──
  oauthAccount?: {
    accountUuid: string
    emailAddress: string
    organizationUuid?: string
    displayName?: string
  }

  // ── MCP Servers ──
  mcpServers?: Record<string, McpServerConfig>

  // ── Hook Configuration ──
  hooks?: HookConfig[]

  // ── Project-Specific ──
  projects?: Record<string, ProjectConfig>
}
```

8.3 CLAUDE.md Memory System

Loading Hierarchy

```
1. Managed memory    /etc/claude-code/CLAUDE.md
2. User memory       ~/.claude/CLAUDE.md
3. Project memory    CLAUDE.md, .claude/CLAUDE.md, .claude/rules/*.md
4. Local memory      CLAUDE.local.md

Note: Files closer to the cwd take higher priority
Maximum character limit: 40,000
```

@include Directive

```
# CLAUDE.md
@path               # Relative path
@./rel/path         # Relative path
@~/home/path        # Home directory
@/abs/path          # Absolute path
```

Features:

- Circular reference detection and prevention
- Only loads text files
- Silently ignores missing files
- Character limit prevents oversized files

8.4 Environment Variables

Variable	Purpose
`CLAUDE_CODE_SIMPLE`	Simplified mode

ANTHROPIC_API_KEY	API key
CLAUDE_CODE_MAX_BUDGET	Maximum budget
CLAUDE_CODE_PERMISSION_MODE	Permission mode
CLAUDE_CODE_VERBOSE	Verbose output

Part 9: Context Management & Compaction

9.1 Token Budget System

```
// Core type
type TokenBudget = {
  maxContextTokens: number       // Model maximum context
  warningThreshold: number       // Warning threshold (80%)
  compactThreshold: number       // Compaction threshold
  estimatedCurrentTokens: number // Estimated current token count
}

// Budget check
calculateTokenWarningState(messages, limits) → {
  isNearLimit: boolean      // Near the limit
  isAtLimit: boolean        // At the limit
  shouldCompact: boolean    // Should compact
  estimatedTokens: number   // Estimated token count
}
```

9.2 Auto-Compaction Mechanism

```
Trigger conditions:
├── Manual: /compact command
├── Automatic: Context > 80% of limit
└── Reactive: After max_output_tokens recovery

Compaction flow:
┌──────────────────────────────────────┐
│ 1. Calculate compaction point        │
│    (compactionIndex)                 │
│    Preserve recent critical messages │
├──────────────────────────────────────┤
│ 2. Generate summary                  │
│    LLM summarizes old messages       │
├──────────────────────────────────────┤
│ 3. Build new message array           │
│    [system_prompt,                   │
│     attachment(summary),             │
│     remaining_messages]              │
└──────────────────────────────────────┘

Snip Compaction (Feature-gated):
├── Replace intermediate messages with minimal summaries
```

```
├── Retain only tool calls and results at boundaries
└── Dramatically reduce token consumption
```

9.3 Prompt Cache Optimization

Claude Code uses **multi-level prompt caching** to reduce redundant token consumption:

```
┌─ System Prompt Structure ──────────────────────┐
│                                                 │
│  ┌──────────────────────────────────────┐       │
│  │ Static portion (cacheable across     │       │
│  │ organizations)                       │       │
│  │ ├─ Base role instructions            │       │
│  │ ├─ Tool definitions                 │       │
│  │ ├─ General guidelines               │       │
│  │ └─ cache_control: ephemeral         │       │
│  ├──────── Dynamic boundary ────────────┤       │
│  │ __SYSTEM_PROMPT_DYNAMIC_BOUNDARY__  │       │
│  ├──────────────────────────────────────┤       │
│  │ Dynamic portion (varies per          │       │
│  │ session)                             │       │
│  │ ├─ User context (CLAUDE.md)          │       │
│  │ ├─ Git status                        │       │
│  │ ├─ Session-specific settings         │       │
│  │ └─ Feature tips                      │       │
│  └──────────────────────────────────────┘       │
│                                                 │
│  Cache effect: ~90% redundant token             │
│  reduction in long conversations                │
└─────────────────────────────────────────────────┘
```

9.4 Section Caching Mechanism

```
// Regular section: computed once, reused for the entire session
systemPromptSection(name, compute): SystemPromptSection

// Uncacheable section: recomputed every turn
DANGEROUS_uncachedSystemPromptSection(name, compute, reason)

// Cache is cleared on /clear and /compact
```

Part 10: Agent System

10.1 Agent Definition

```
type AgentDefinition = {
  agentType: string          // Agent type identifier
  name: string               // Display name
  description: string        // Description
  whenToUse: string          // Usage scenario description
```

```
  // Capability configuration
  tools?: string[]           // Allowed tools (whitelist)
  disallowedTools?: string[] // Forbidden tools (blacklist)

  // Persona settings
  getSystemPrompt(ctx?): string  // System prompt
  memory?: string                // Memory type

  // Execution configuration
  model?: string                 // Model to use
  thinkingConfig?: ThinkingConfig // Thinking configuration
}
```

10.2 Built-in Agent Types

Agent	Function	Tool Restrictions
general-purpose	General autonomous agent	All tools
Plan	Architecture planning agent	Read-only tools, no edit/write
Explore	Codebase exploration agent	Read-only search tools
claude-code-guide	Help/tutorial agent	Glob, Grep, Read, WebFetch, WebSearch
code-reviewer	Code review agent	Read-only tools

10.3 Agent Execution Model

```
Main Agent
  │
  ├── Agent(subagent_type: "Explore")
  │     └── Independent context, read-only tools
  │         Returns research results
  │
  ├── Agent(subagent_type: "general-purpose")
  │     └── Full context, all tools
  │         Autonomously completes complex tasks
  │
  └── Agent(isolation: "worktree")
        └── Git worktree isolation
            Independent repository copy
            Merge or clean up upon completion
```

10.4 Agent Prompt Best Practices

Prompt guidelines embedded in the source code:

> **Brief the agent like a smart colleague who just walked into the room** — they haven't seen this conversation, don't know what you've tried, and don't understand why this task matters.
>
> - Explain what you want to accomplish and why

- Describe what you've already learned or ruled out
- Provide enough context for the agent to make judgments
- If you need a brief response, say so explicitly

10.5 Custom Agents

Users can create custom agents in the `~/.claude/agents/` directory:

```
// ~/.claude/agents/security-reviewer.json
{
  "agentType": "security-reviewer",
  "name": "Security Reviewer",
  "description": "An agent specialized in reviewing code security",
  "whenToUse": "When a security review is needed",
  "tools": ["FileRead", "Glob", "Grep", "Bash"],
  "disallowedTools": ["FileEdit", "FileWrite"],
  "systemPrompt": "You are a security expert specializing in reviewing
}
```

Part 11: System Prompt Construction

11.1 Prompt Build Flow

```
buildEffectiveSystemPrompt({
  mainThreadAgentDefinition?: AgentDefinition
  toolUseContext: ToolUseContext
  customSystemPrompt?: string
  defaultSystemPrompt: string[]
  appendSystemPrompt?: string
  overrideSystemPrompt?: string
})
```

Priority chain:

1. **Override prompt** — Replaces everything (e.g., /loop mode)
2. **Coordinator mode prompt** — Multi-agent coordination
3. **Agent system prompt** — If mainThreadAgentDefinition is set
4. **Custom prompt** — User-specified via `--system-prompt`
5. **Default prompt** — Generated from `prompts.ts`

11.2 Default System Prompt Sections

Section	Purpose	Content
`getSimpleIntroSection()`	Role introduction	"You are an interactive agent..."
`getSimpleSystemSection()`	System description	Tools, permissions, labels, hooks, compaction
`getSimpleDoingTasksSection()`	Task guidance	Software engineering best practices, code style, security

```
// ├─ Streaming response handling
// ├─ Error classification
// └─ Retry coordination
```

12.2 Exponential Backoff Retry Strategy

```
Attempt 1: Execute immediately
Attempt 2: Wait 1s
Attempt 3: Wait 2s
Attempt 4: Wait 4s
Attempt 5: Wait 8s (max 5 retries)

Retryable errors:
├── 429 (Rate limit)
├── 500 (Server error)
├── 502, 503, 504 (Gateway errors)
└── Transient network errors

Non-retryable errors:
├── 401 (Authentication failure)
├── 403 (Insufficient permissions)
├── 400 (Bad request)
└── 413 (Payload too large)
```

12.3 max_output_tokens Recovery Mechanism

When the API returns `stop_reason: "max_tokens"`:

```
max_output_tokens detected
     │
     ▼
┌──────────────────────────┐
│ Increment recovery       │
│ counter                  │
│ maxOutputTokensRecovery  │
│ Count++                  │
├──────────────────────────┤
│ Upgrade token budget     │
│ Request higher output    │
│ limit                    │
├──────────────────────────┤
│ If too many recoveries:  │
│ Attempt reactive         │
│ compaction               │
│ hasAttemptedReactive     │
│ Compact = true           │
├──────────────────────────┤
│ Re-enter query loop      │
└──────────────────────────┘
```

Part 13: Build System & Compilation Optimization

13.1 Five-Stage Build Pipeline

```
Stage 1: Copy
   src/ → build-src/ (preserve original files)

Stage 2: Transform
   ├─ feature('X') → false (dead code marking)
   ├─ MACRO.VERSION → '2.1.88' (version injection)
   ├─ import from 'bun:bundle' → stubs (Bun features)
   └─ Remove type-only imports

Stage 3: Entry Wrapping
   Create build-src/entry.ts → points to src/entrypoints/cli.tsx

Stage 4: Iterative Stubbing + Bundling
   Round N:
   ├─ 1. esbuild bundles entry.ts
   ├─ 2. Collect missing modules
   ├─ 3. Create stubs for missing modules
   └─ 4. Repeat until successful

Stage 5: Output
   Generate final cli.js + cli.js.map
```

13.2 Feature Gate Compile-Time Elimination

```
// In source code
if (feature('DAEMON')) {
  require('./daemon/main.js')
}

// After compilation (public release)
if (false) {
  // Entire branch eliminated by esbuild (dead code elimination)
}
```

This mechanism causes **108 modules** to be eliminated in the public release.

13.3 Eliminated Internal Features (108 Modules)

Internal infrastructure (~70):

- `daemon/main.js` — Background monitor
- `coordinator/workerAgent.js` — Multi-agent coordination
- `assistant/` — KAIROS autonomous mode
- `bridge/peerSessions.js` — Remote control peer management
- `skillSearch/` — Remote skill discovery
- `sessionTranscript/sessionTranscript.js` — Classifier backend

Feature-gated tools (~20):

- `REPLTool` — Interactive REPL
- `MonitorTool` — MCP monitoring
- `WebBrowserTool` — Browser automation
- `WorkflowTool` — Workflow execution
- `SnipTool` — Context snipping
- `SleepTool` — Delay/scheduling

Part 14: Power User Tips (Beginner)

14.1 Basic Configuration Optimization

Creating a CLAUDE.md Project File

Create a `CLAUDE.md` in your project root — this is the **single most important step** to improving Claude Code's effectiveness:

```
# Project Description

## Tech Stack
- Next.js 14 + TypeScript
- Prisma + PostgreSQL
- Tailwind CSS

## Development Commands
- `npm run dev` — Start dev server
- `npm test` — Run tests
- `npm run lint` — Lint code
- `npm run build` — Build project

## Code Conventions
- Use function components + Hooks
- State management with Zustand
- API routes in /app/api/ directory
- Components in /components/ directory

## Database
- Schema in prisma/schema.prisma
- Migration command: npx prisma migrate dev

## Important Notes
- Do not modify files under /lib/legacy/ — that is legacy code
- Tests must pass before committing
- Use Chinese comments
```

Why this matters: Source code analysis shows that CLAUDE.md content is injected into the system prompt on **every turn**, directly influencing the model's behavior.

Setting Up Permission Rules

```
// ~/.claude/settings.json
{
  "permissions": {
    "rules": {
      "allow": [
        "FileRead",
        "FileEdit",
        "Glob",
        "Grep",
        "Bash(prefix:npm test)",
        "Bash(prefix:npm run lint)",
        "Bash(prefix:git status)",
        "Bash(prefix:git diff)",
        "Bash(prefix:git log)"
      ],
      "deny": [
        "Bash(prefix:rm -rf)",
        "Bash(prefix:sudo)",
        "Bash(prefix:git push --force)"
      ]
    }
  }
}
```

14.2 Slash Command Quick Reference

Command	Function	When to Use
`/help`	Show help information	When unsure about features
`/compact`	Compact context	When conversations get too long
`/clear`	Clear conversation history	When starting a new topic
`/status`	Show session status	Check token usage
`/cost`	Show cost statistics	Track API spending
`/model`	Switch model	Adjust speed/quality tradeoff
`/commit`	Git commit	Auto-generate commit messages
`/review`	Code review	Review PRs or changes
`/mcp`	MCP management	Add/remove MCP servers
`/config`	Configuration management	Modify settings
`/resume`	Resume session	Continue previous work
`/tasks`	Task management	View/manage task list
`/permissions`	Permission management	Adjust permission rules
`/hooks`	Hook management	View/configure hooks

14.3 Basic Usage Patterns

Pattern 1: Exploratory Programming

```
You: Help me understand this project's structure, focusing on API route

Claude Code will:
├── Use Glob to search the file structure
├── Use Grep to find API definitions
├── Use Read to examine key files
└── Generate a project structure overview
```

Pattern 2: Bug Fixing

```
You: The page goes blank after user login — please investigate and fix

Claude Code will:
├── Use Grep to search login-related code
├── Use Read to examine related components
├── Use Bash to run tests
├── Use Edit to fix the bug
└── Use Bash to verify the fix
```

Pattern 3: Feature Development

```
You: Add a user avatar upload feature with cropping support

Claude Code will:
├── Analyze existing code structure
├── Create necessary new files
├── Modify related components
├── Write API routes
├── Add tests
└── Run tests to verify
```

14.4 Shell Command Integration Tips

Use the `!` prefix in the Claude Code prompt to run interactive commands:

```
You: ! gcloud auth login

# Claude Code will execute the command in the current shell
# Output flows directly into the conversation context
```

Part 15: Power User Tips (Intermediate)

15.1 Leveraging the Agent System for Parallel Work

Source code analysis reveals that the Agent tool supports spawning multiple sub-agents in parallel. Take advantage of this:

```
You: Please do the following tasks simultaneously:
    1. Review the security of the src/auth/ directory
    2. Write unit tests for src/api/users.ts
    3. Check the entire project for N+1 query issues

# Claude Code will spawn 3 sub-agents in parallel
# Each sub-agent works independently without interference
# Results are aggregated and returned together
```

Key finding: The source code in `AgentTool.tsx` shows that sub-agents can be set to `isolation: "worktree"` to run in isolated Git worktrees, preventing file conflicts.

15.2 Deep CLAUDE.md Utilization

Multi-Level CLAUDE.md

```
~/.claude/CLAUDE.md                    # Global preferences
├── "I am a senior TypeScript developer"
├── "Prefer functional programming style"
└── "Reply in Chinese"

project-root/CLAUDE.md                 # Project-level config
├── "Use pnpm instead of npm"
├── @./docs/architecture.md            # Reference architecture docs
└── @./docs/api-spec.md               # Reference API spec

project-root/.claude/rules/security.md # Security rules
├── "All user input must be escaped"
└── "Do not use eval() or innerHTML"

project-root/.claude/rules/testing.md  # Testing rules
├── "All new features must have unit tests"
└── "Test coverage must not drop below 80%"
```

@include in Practice

```
# CLAUDE.md

## Project Documentation
@./README.md
@./docs/CONTRIBUTING.md
@./docs/API.md

## Code Standards
@./.eslintrc.json
@./tsconfig.json
```

15.3 Extending Capabilities with MCP Servers

Common MCP Server Configurations

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "."]
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
    },
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": { "DATABASE_URL": "${DATABASE_URL}" }
    },
    "slack": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-slack"],
      "env": { "SLACK_TOKEN": "${SLACK_TOKEN}" }
    }
  }
}
```

15.4 Hook-Based Workflow Automation

Auto-Format Code

```
{
  "hooks": [
    {
      "event": "PostToolUse",
      "matcher": "FileEdit|FileWrite",
      "handler": {
        "type": "script",
        "command": "npx prettier --write \"$TOOL_INPUT_FILE_PATH\" 2>/d
      }
    }
  ]
}
```

Auto-Run Tests

```
{
  "hooks": [
    {
      "event": "PostToolUse",
      "matcher": "FileEdit",
      "handler": {
        "type": "script",
```

```
        "command": "if echo \"$TOOL_INPUT_FILE_PATH\" | grep -q '\\.tes
      }
    }
  ]
}
```

15.5 Session Management Tips

Using /resume to Restore Work

```
# View previous sessions
You: /resume

# Resume a specific session
You: /resume [session-id]

# Claude Code will load the complete conversation history and context
```

Controlling Context Size

```
# When a conversation gets too long
You: /compact

# More aggressive compaction
You: /compact --aggressive

# Clear when starting a new topic
You: /clear
```

15.6 Prompt Engineering Tips

After analyzing the system prompt from the source code, here are the most effective prompt patterns:

Precise Instruction Pattern

```
BAD:  "Help me fix this file"
GOOD: "Modify the getUser function in src/api/users.ts,
       change the database query from findFirst to findUnique,
       because the ID is unique"
```

Context Preloading Pattern

```
You: First read src/models/user.ts and src/api/users.ts,
     then tell me how the user data flows from API to database

# This leverages the fact that the Read tool is more efficient
# than Bash(cat) as revealed in the source code
```

Multi-Step Task Decomposition Pattern

```
You: I need to implement a user export feature. Please follow these ste
    1. First analyze the existing data models
    2. Design the API interface
    3. Implement the backend logic
    4. Write tests
    After completing each step, tell me your plan and wait for
    my confirmation before proceeding
```

Part 16: Power User Tips (Advanced)

16.1 Deep Understanding of the Permission System

Permission decision reason types (PermissionDecisionReason) discovered in the source code:

```
type PermissionDecisionReason =
  │ { type: 'classifier' }    // Auto mode classifier decision
  │ { type: 'hook' }          // Hook decision
  │ { type: 'rule' }          // Permission rule match
  │ { type: 'mode' }          // Permission mode decision
  │ { type: 'safetyCheck' }   // Safety check
  │ { type: 'sandboxOverride' } // Sandbox override
  │ { type: 'workingDir' }    // Working directory check
  │ { type: 'asyncAgent' }    // Async agent
```

Practical application: Understanding these decision types allows you to precisely configure rules to optimize the approval flow.

16.2 Token Budget Management Strategies

```
Strategy 1: Budget-Aware Programming
├── In large projects, prefer Glob + Grep over full-file Read
├── Provide precise line ranges: Read(file, offset=100, limit=50)
├── Use /compact at key checkpoints to compress context
└── Split complex tasks into multiple smaller sessions

Strategy 2: Understanding Auto-Compact Triggers
├── Automatically triggers when context reaches 80% of model limit
├── Compaction loses some early context details
├── Use /compact manually before important context gets compacted
└── Monitor /status or /cost to track current token usage

Strategy 3: Prompt Cache Utilization
├── Keep CLAUDE.md content stable (cache-friendly)
├── Long conversations reuse the cached portion of the system prompt
└── Avoid frequent config changes that invalidate the cache
```

16.3 Building Complex Hook Pipelines

Agent Hook — AI-Driven Code Review

```
{
  "hooks": [
    {
      "event": "PreToolUse",
      "matcher": "FileEdit",
      "handler": {
        "type": "agent",
        "prompt": "Review the upcoming file edit. Check for:\n1. Securi
        "timeout": 30,
        "model": "claude-haiku-4-5-20251001"
      }
    }
  ]
}
```

Multi-Stage Validation Pipeline

```
{
  "hooks": [
    {
      "event": "PostToolUse",
      "matcher": "FileEdit",
      "handler": {
        "type": "script",
        "command": "bash -c 'npx eslint --fix \"$TOOL_INPUT_FILE_PATH\"
      }
    },
    {
      "event": "PostToolUse",
      "matcher": "Bash",
      "handler": {
        "type": "javascript",
        "code": "module.exports = async ({input}) => { if (input.comman
      }
    }
  ]
}
```

16.4 Using Worktree Isolation for Experiments

The `worktree` isolation mode discovered in the source code:

```
You: Please try migrating the ORM from Prisma to Drizzle in an isolated
     environment — don't affect the current code

# Claude Code will:
# 1. Create a Git worktree (independent repository copy)
# 2. Execute all modifications in the worktree
# 3. If successful, return the worktree path and branch name
```

```
# 4. If it fails, automatically clean up the worktree
```

16.5 SDK Mode Integration

```
// Call Claude Code from your own scripts
import { QueryEngine } from '@anthropic-ai/claude-code';

const engine = new QueryEngine({
  cwd: '/path/to/project',
  tools: getAllBaseTools(),
  // ... configuration
});

// Submit a query and receive streaming results
for await (const event of engine.submitMessage('Analyze project depende
  if (event.type === 'message') {
    console.log(event.content);
  }
}
```

16.6 Performance Optimization Tips

Tip	Rationale	Impact
Pre-populate CLAUDE.md	Injected into system prompt, reduces exploration overhead	-30% tokens
Use specific file paths	Avoids Glob searches	Faster responses
Batch large tasks	Use /compact to control context	More stable
Configure allow rules	Reduces permission prompt interactions	Smoother flow
Use parallel Agents	Sub-agents work independently in parallel	Higher throughput
Leverage MCP tools	Extend the tool set	Greater capabilities

16.7 Debugging & Diagnostics

```
# Enable verbose mode
Set "verbose": true in settings.json

# View session status
/status

# View token usage and costs
/cost

# Analyze context window usage
# (analyzeContext.ts in the source provides this functionality)

# View current configuration
```

```
/config

# Diagnose issues
/doctor
```

Part 17: Hidden Internal Features & Feature Gates

17.1 Feature Gate Mechanism

The source code uses `feature('FEATURE_NAME')` for compile-time conditional compilation:

```
// In public release: feature('X') → false
// In internal builds: feature('X') → true (evaluated at Bun compile ti
```

17.2 Known Feature Gate List

Feature Gate	Function	Status
`DAEMON`	Background daemon	Internal
`BRIDGE_MODE`	Remote control bridge	Internal
`BG_SESSIONS`	Background sessions	Internal
`COORDINATOR_MODE`	Multi-agent coordination	Internal
`KAIROS`	Fully autonomous assistant mode	Internal
`PROACTIVE`	Proactive agent	Internal
`TRANSCRIPT_CLASSIFIER`	Classifier	Internal
`AGENT_TRIGGERS`	Scheduled triggers	Partially open
`AGENT_TRIGGERS_REMOTE`	Remote triggers	Internal
`DIRECT_CONNECT`	Direct connection	Internal
`SSH_REMOTE`	SSH remote	Internal
`HISTORY_SNIP`	History snipping	Internal
`BREAK_CACHE_COMMAND`	Cache clearing command	Internal
`CHICAGO_MCP`	Computer Use MCP	Internal
`DUMP_SYSTEM_PROMPT`	Dump system prompt	Internal
`WORKFLOW_SCRIPTS`	Workflow scripts	Internal

17.3 Undisclosed Internal Tools

Tool	Function	Restriction
`REPLTool`	Interactive REPL	ANT-only
`MonitorTool`	MCP monitoring	Feature-gated
`WebBrowserTool`	Browser automation	Feature-gated
`WorkflowTool`	Workflow execution	Feature-gated

`SnipTool`	Context snipping	Feature-gated
`SleepTool`	Delay/scheduling	Feature-gated
`TerminalTool`	Terminal control	Feature-gated
`TeamCreateTool`	Team creation	Feature-gated
`TeamDeleteTool`	Team deletion	Feature-gated

17.4 KAIROS Autonomous Mode

KAIROS system information extracted from the source code:

```
KAIROS (Fully Autonomous Assistant Mode):
├── Always-running background AI agent
├── Push notification support
├── GitHub PR webhook integration
├── <tick> heartbeat mechanism — periodic autonomous execution
├── Push-to-talk voice mode
├── Proactive tools (SleepTool, CronTools)
└── Standalone assistant/ module system
```

17.5 Coordinator Multi-Agent Mode

```
Coordinator Mode:
├── Maintains independent conversation context for each agent
├── Dispatches work based on best-fit matching
├── Aggregates multi-agent results
├── workerAgent.js — Worker agent
└── coordinatorMode.js — Coordinator core
```

Part 18: Architectural Design Patterns Summary

18.1 Async Generator Streaming

```
// Core pattern: streaming via async generators
async function* query(params): AsyncGenerator<StreamEvent | Message> {
  // Yield individual events
  // Allows consumers to respond to partial results in real time
  // Resource cleanup via return()
}

// Consumption pattern
for await (const event of query(params)) {
  if (event.type === 'tool_call') { /* update UI */ }
  if (event.type === 'message') { /* display text */ }
}
```

18.2 Feature Gate Compile-Time Elimination

```
// Compile-time conditional compilation + dead code elimination
if (feature('FEATURE_NAME')) {
  // Bun: included when flag=true, DCE'd when flag=false
  // esbuild: feature() → false, entire branch eliminated
  require('./feature-module.js')
}
```

18.3 Lazy Loading to Break Circular Dependencies

```
// Avoid circular module references
const getTeammateUtils = () =>
  require('./utils/teammate.js') as typeof import('./utils/teammate.js'

// Only loaded when actually called
const utils = getTeammateUtils()
```

18.4 Memoization for Expensive Operations

```
export const getSystemContext = memoize(async () => {
  // Cached for the entire session
  // Cleared on settings changes
})
```

18.5 Discriminated Unions for Type Safety

```
type Message =
  | { type: 'user'; content: string }
  | { type: 'assistant'; message: { content: ContentBlockParam[] } }
  | { type: 'attachment'; metadata: { name: string } }

// Compile-time exhaustiveness checking
if (message.type === 'user') { ... }
else if (message.type === 'assistant') { ... }
// TypeScript ensures all types are handled
```

18.6 Parallel Prefetching for Startup Optimization

```
// Parallel prefetch pattern in main.tsx
startMdmRawRead()        // Parallel 1: MDM subprocess
startKeychainPrefetch()  // Parallel 2: Keychain read

// Executes in parallel with module loading (~135ms)
// Results awaited on demand when needed
```

18.7 Dual-Track State Management

```
React State (AppStateStore):
├── Used by interactive UI
├── Lifecycle managed by React
└── For REPL mode
```

```
Bootstrap State (bootstrap/state.ts):
├── Global mutable state
├── Available before React renders
└── For SDK/headless mode
```

Part 19: Key Differences from the Standard Claude API

Feature	Standard Claude API	Claude Code
Execution	API calls	CLI + interactive TUI
File access	None	Full file system
Code execution	None	Bash/PowerShell
Session state	Stateless	Persistent session storage
Tool permissions	Fixed API key	Interactive prompts + multi-mode
Context management	Fixed window	Auto-compaction + caching
Agent system	Not built-in	Full sub-agent/coordinator
MCP support	Not built-in	Full MCP client + registry
Plugins/Skills	Not supported	Full plugin system
Git integration	Not supported	Deep Git state + hooks
CLAUDE.md	Not supported	Multi-level support
Security model	None	Rules + classifier + sandbox
Cost control	Per-call	Token budget + auto-compaction
Multi-model	Manual selection	Dynamic switching + fallback

Part 20: Future Roadmap & Outlook

20.1 Known Development Directions

Future directions inferred from feature gates and code comments in the source:

```
1. KAIROS Fully Autonomous Mode
   ├── Always-on background AI assistant
   ├── Proactive task execution
   └── Voice interaction support

2. Coordinator Multi-Agent Orchestration
   ├── Multiple specialized agents collaborating
   ├── Automatic task assignment
   └── Result aggregation and conflict resolution

3. Stronger MCP Ecosystem
   ├── Official MCP registry
   ├── Full OAuth support
   └── More transport protocols
```

```
4. Context Management Evolution
   ├── Snip compaction (smarter context pruning)
   ├── Context Collapse
   └── Support for larger context windows

5. Enterprise Features
   ├── MDM policy management
   ├── Remote settings push
   ├── Compliance audit logging
   └── SSO integration
```

20.2 Model Iteration Cadence

Model launch annotations `@[MODEL LAUNCH]` in the source code show:

- `FRONTIER_MODEL_NAME = 'Claude Opus 4.6'` — Current frontier model
- Each model update requires modifying model names in multiple prompt locations
- Supports Bedrock and Vertex AI multi-platform deployment

Appendix

Appendix A: Complete Tool Parameter Reference

FileRead

```
{
  "file_path": "Absolute path (required)",
  "offset": "Starting line number (optional)",
  "limit": "Number of lines to read (optional, default 2000)",
  "pages": "PDF page range (optional, e.g. '1-5')"
}
```

FileEdit

```
{
  "file_path": "Absolute path (required)",
  "old_string": "Text to replace (required, must be unique)",
  "new_string": "Replacement text (required)",
  "replace_all": "Replace all matches (optional, default false)"
}
```

Bash

```
{
  "command": "Shell command (required)",
  "timeout": "Timeout in milliseconds (optional, default 120000, max 60
  "run_in_background": "Run in background (optional, default false)"
```

```
}
```

Agent

```
{
  "prompt": "Task description (required)",
  "description": "3-5 word short description (required)",
  "subagent_type": "Agent type (optional)",
  "model": "Model override (optional, sonnet|opus|haiku)",
  "isolation": "Isolation mode (optional, 'worktree')",
  "run_in_background": "Run in background (optional)"
}
```

Grep

```
{
  "pattern": "Regular expression (required)",
  "path": "Search directory (optional, default cwd)",
  "glob": "File filter (optional, e.g. '*.ts')",
  "type": "File type (optional, e.g. 'js', 'py')",
  "output_mode": "Output mode (optional, content|files_with_matches|cou
  "context": "Context lines (optional)",
  "head_limit": "Result limit (optional, default 250)",
  "multiline": "Multiline matching (optional, default false)"
}
```

Glob

```
{
  "pattern": "Glob pattern (required, e.g. '**/*.tsx')",
  "path": "Search directory (optional, default cwd)"
}
```

Appendix B: Complete settings.json Template

```
{
  "theme": "dark",
  "verbose": false,
  "autoCompactEnabled": true,

  "permissions": {
    "defaultMode": "default",
    "rules": {
      "allow": [
        "FileRead",
        "FileEdit",
        "FileWrite",
        "Glob",
        "Grep",
        "Bash(prefix:npm)",
```

```
        "Bash(prefix:git status)",
        "Bash(prefix:git diff)",
        "Bash(prefix:git log)",
        "Bash(prefix:ls)",
        "Bash(prefix:pwd)"
      ],
      "deny": [
        "Bash(prefix:rm -rf /)",
        "Bash(prefix:sudo)",
        "Bash(prefix:git push --force)"
      ],
      "ask": []
    }
  },

  "env": {
    "NODE_ENV": "development"
  },

  "mcpServers": {},

  "hooks": [],

  "preferredNotifChannel": "system"
}
```

Appendix C: CLAUDE.md Best Practices Template

```
# Project: [Project Name]

## Overview
[One-sentence project description]

## Tech Stack
- Language: [TypeScript/Python/...]
- Framework: [Next.js/FastAPI/...]
- Database: [PostgreSQL/MongoDB/...]
- Package Manager: [npm/pnpm/poetry/...]

## Directory Structure
- src/ — Source code
- tests/ — Tests
- docs/ — Documentation
- scripts/ — Scripts

## Development Commands
- Start: `npm run dev`
- Test: `npm test`
- Build: `npm run build`
- Lint: `npm run lint`
```

```
## Coding Standards
- [List key standards]

## Important Notes
- [List items requiring special attention]

## Referenced Documentation
@./docs/architecture.md
@./docs/api.md
```

Appendix D: Key Source File Index

Function	File Path	Lines
CLI entry	`src/entrypoints/cli.tsx`	~300
Main orchestrator	`src/main.tsx`	~4,683
Query engine	`src/QueryEngine.ts`	~200+
Core loop	**`src/query.ts`**	**~1,729**
Tool definition	`src/Tool.ts`	~200+
Tool registration	`src/tools.ts`	~300+
Streaming executor	`src/services/tools/StreamingToolExecutor.ts`	~200+
Permission engine	`src/utils/permissions/permissions.ts`	~250+
YOLO classifier	`src/utils/permissions/yoloClassifier.ts`	52KB
Hook execution	`src/utils/hooks/execAgentHook.ts`	~200+
Hook types	`src/types/hooks.ts`	~300+
MCP client	`src/services/mcp/client.ts`	119KB
MCP types	`src/services/mcp/types.ts`	~200+
API client	`src/services/api/claude.ts`	125KB
Retry logic	`src/services/api/withRetry.ts`	28KB
Context compaction	`src/services/compact/compact.ts`	~100+
Auto-compaction	`src/services/compact/autoCompact.ts`	~100+
Config loading	`src/utils/config.ts`	~250+
CLAUDE.md	`src/utils/claudemd.ts`	~200+
Prompt construction	`src/constants/prompts.ts`	~300+
Prompt caching	`src/constants/systemPromptSections.ts`	~60
System prompt	`src/utils/systemPrompt.ts`	~124
Message types	`src/types/message.ts`	~100+
Permission types	`src/types/permissions.ts`	~250+
App state	`src/state/AppStateStore.ts`	~150+
Command registration	`src/commands.ts`	~200+
Token counting	`src/utils/tokens.ts`	~100+
Context analysis	`src/utils/analyzeContext.ts`	42KB

Build script	`scripts/build.mjs`	~230
Agent tool	`src/tools/AgentTool/AgentTool.tsx`	233KB
Agent loading	`src/tools/AgentTool/loadAgentsDir.ts`	26KB

Appendix E: Frequently Asked Questions

Q1: How large is Claude Code's context window?
A: It depends on the model in use. Claude Opus 4.6 supports a 1M token context. The system automatically manages context and triggers compaction as it approaches the limit.

Q2: How can I reduce API costs?
A:

1. Use CLAUDE.md to reduce exploratory token consumption
2. Use specific file paths instead of searching
3. Use /compact promptly to compress context
4. Use the Haiku model for simple tasks

Q3: What is the difference between hooks and MCP?
A: Hooks inject logic at internal Claude Code event points (e.g., before/after tool calls), while MCP extends available external tools and data sources through a standard protocol.

Q4: How many levels deep can sub-agents be nested?
A: There is no hard limit in the source code, but each level of sub-agent consumes token budget. In practice, it is recommended not to exceed 2-3 levels.

Q5: How much content can CLAUDE.md hold?
A: The source code defines `MAX_MEMORY_CHARACTER_COUNT = 40,000` characters. Content beyond this limit is truncated.

Acknowledgments

This whitepaper is based on source code analysis from the following public repositories:

- sanbuphy/claude-code-source-code
- ChinaSiro/claude-code-sourcemap

All rights reserved by the author. For technical learning and research purposes only.